# Tenant Configuration

*defaults, overrides, settings, and entitlements by name*

Watch the 4-min video

**In one line:** model per-tenant behaviour as **layered configuration** (plan defaults, tenant overrides, tenant settings), have code check **named entitlements** rather than plan names, enforce them on the server, cache the effective config with invalidation, and audit every change.

## 1 The deal that becomes an if statement

Sales closes a deal: this enterprise customer gets advanced analytics, SSO, seven-year data retention and a higher API limit. A starter customer gets none. If each difference becomes `if customer == 'acme'` or `if plan == 'enterprise'` in code, every deal is a deployment, every pricing change is a code search, and nobody can say what a given tenant actually has. Tenant configuration turns those differences into data.

## 2 Configuration in layers

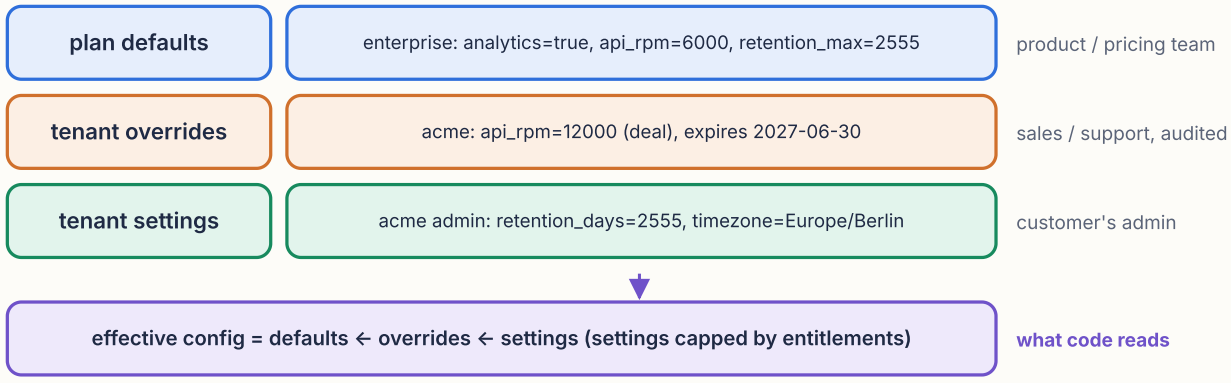



**Figure 1.** Three layers owned by three different people, merged into one effective configuration per tenant.

- **Plan defaults** define what each tier includes (starter, business, enterprise). Owned by product and pricing; changing a plan changes every tenant on it.
- **Tenant overrides** capture deals and exceptions: a custom limit, a trial of a higher-tier feature, an early-access grant. Each has an owner, a reason and, ideally, an **expiry date**; overrides without expiry quietly become permanent.
- **Tenant settings** are choices the customer's admin makes: logo, time zone, password policy, retention period. Settings live inside the bounds of entitlements: an admin can choose retention up to the entitled maximum, not beyond it.

The **effective configuration** is computed by merging the layers, and that's the only thing application code reads.

## 3 Entitlements: check capabilities, not plans

Entitlements are what a tenant has paid for. They come in three shapes: **booleans** (`advanced_analytics`, `sso`), **limits** (`api_rpm`, `seats`, `retention_days_max`), and **enumerations** (support tier). Code asks for the

capability by name. Then moving SSO from enterprise to business is a data change, a custom deal needs no deployment, and a new plan is a new row.

```
# never:   if tenant.plan == 'enterprise': ...
# instead: ask for the capability by name
cfg = tenant_config(ctx.tenant)              # cached, effective config
if not cfg.entitled('advanced_analytics'):
    raise Forbidden(code='upgrade_required', feature='advanced_analytics')
limit = cfg.limit('api_rpm')                 # numeric entitlement
```

**Figure 2.** Check the named entitlement; return a machine-readable error the UI can turn into an upgrade prompt.

Limits connect to other lessons: `api_rpm` feeds the per-tenant rate limiter (lesson 44), quota limits feed metering and billing (lesson 45), and billing systems such as Stripe can be the source of which plan a tenant is on, with your config service deriving entitlements from it.

## 4 Entitlements vs feature flags

| | Feature flag | Entitlement |
|---|---|---|
| Question | is this feature ready, and for whom? | has this tenant paid for it? |
| Owner | engineering / product | product, sales, billing |
| Lifetime | temporary: removed after rollout | permanent part of the pricing model |
| Typical targeting | % of tenants, internal users, a region | plan, overrides, contract |
| If wrong | bug exposure | revenue leak or angry customer |




flag off → hidden (not ready yet) · flag on, not entitled → upgrade prompt · both → available

*e.g. an AI-summaries beta: flag on for 5% of tenants, entitlement only in business and enterprise*

**Figure 3.** A feature reaches a tenant only when the flag is on and the tenant is entitled.

## 5 Caching, invalidation and storage

Configuration is read on almost every request, so each service caches the effective config in memory. Two mechanisms together: **change events** (the config service publishes `tenant acme changed, version 42` and services evict the entry, so an upgrade takes effect in seconds) and a short **TTL** (say 60 seconds) that bounds staleness if an event is lost. If the config service is down, keep serving the **last-known-good** config rather than failing every request.

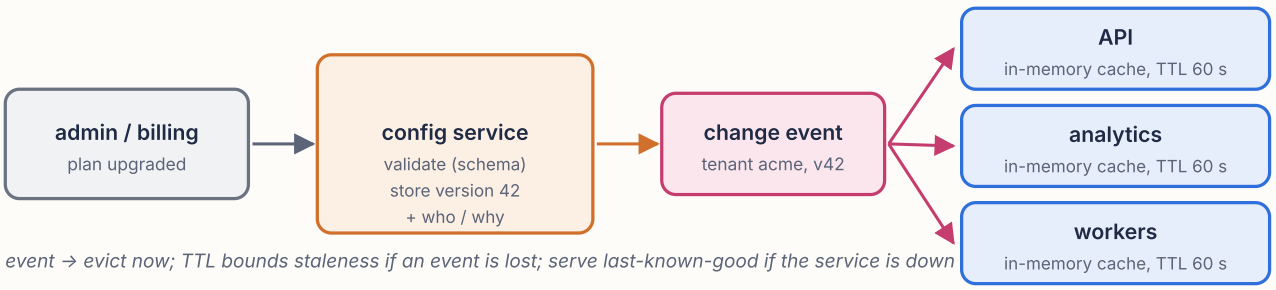



**Figure 4.** A central, validated, versioned config service, with services caching the effective config and evicting it on change.

Store configuration centrally with a **schema** (reject `api_rpm = "lots"`), **versions** and **history**: every change records who made it, when and why, and lands in the audit trail (lesson 48). Manual overrides by your own staff deserve an approval step. And **enforce on the server**: the UI can hide a button, but the API must refuse the call, because customers call your API directly.

**WORKED EXAMPLE: AN ENTERPRISE-ONLY FEATURE WITH A TRIAL**

`advanced_analytics` is `true` in the enterprise plan defaults and `false` elsewhere. A business customer gets a 90-day trial through a tenant override `advanced_analytics = true, expires_at = 2027-02-01`. The analytics service checks the entitlement from its cached effective config. Expiry needs no cron job: the merge ignores overrides whose `expires_at` has passed, so the next evaluation after the date returns false (within the cache TTL). The API then answers `403 upgrade_required` and the UI shows an upgrade prompt, while the tenant's data is kept, so upgrading later restores the dashboards.

**COMMON MISTAKES**

- Checking plan names or customer names in code.
- Overrides without expiry or owner, so nobody knows why acme has 10× the limit.
- Enforcing entitlements only in the UI.
- Using one flag system for both rollout and paid access, then deleting a "flag" that was really a product tier.

## 6 Key terms

**Plan defaults**: what each tier includes

**Tenant override**: per-tenant exception, ideally with expiry

**Tenant setting**: a choice the customer's admin makes

**Entitlement**: a feature or limit the tenant has paid for

**Effective config**: the merged result code reads

**Feature flag**: temporary switch controlling rollout

## 7 Interview questions

**Q1. How do you manage per-tenant features?** BASIC

Layer plan defaults, tenant overrides and settings; check named entitlements, not plan names; enforce them on the server; cache the effective config with invalidation; audit every change.

**Q2. How are entitlements different from feature flags?** BASIC

Flags control rollout and are temporary; entitlements control what a tenant has paid for and are part of the pricing model.

**Q3. Product wants to move SSO from enterprise to business. What changes?** SCENARIO

Only data: set the sso entitlement to true in the business plan defaults; caches refresh via change events and no deployment is needed.

**Q4. A customer upgrades but still sees the old limits for ten minutes. Why, and how do you fix it?** SCENARIO

Services are caching the old effective config with a long TTL; publish change events that evict the tenant's entry and keep the TTL short as a backstop.

**Q5. The config service goes down. What should services do?** TRICKY

Keep using the last-known-good cached config, rather than failing requests or falling back to the most restrictive defaults.

**Q6. How do you expire a trial without a scheduled job?** TRICKY

Store expires_at on the override and ignore expired overrides when merging, so the effective config changes on the next evaluation.

**Q7. Why must entitlements be enforced in the API and not just the UI?** TRICKY

Customers and scripts call the API directly; a hidden button doesn't stop an unentitled request.

Next: SD-R-010 · Deployment Stamps backendroadmap.com